

Principles of Data Communications and Networks Modeling (CS-6865)

Lab Report #3: Investigation of network congestion 1

Abid Hasan Muin (3804634)

Due date: October 28, 2025

Exercise 1: Plotting throughput with Xgraph, custom trace files

1.1 Screenshot of Xgraph Output

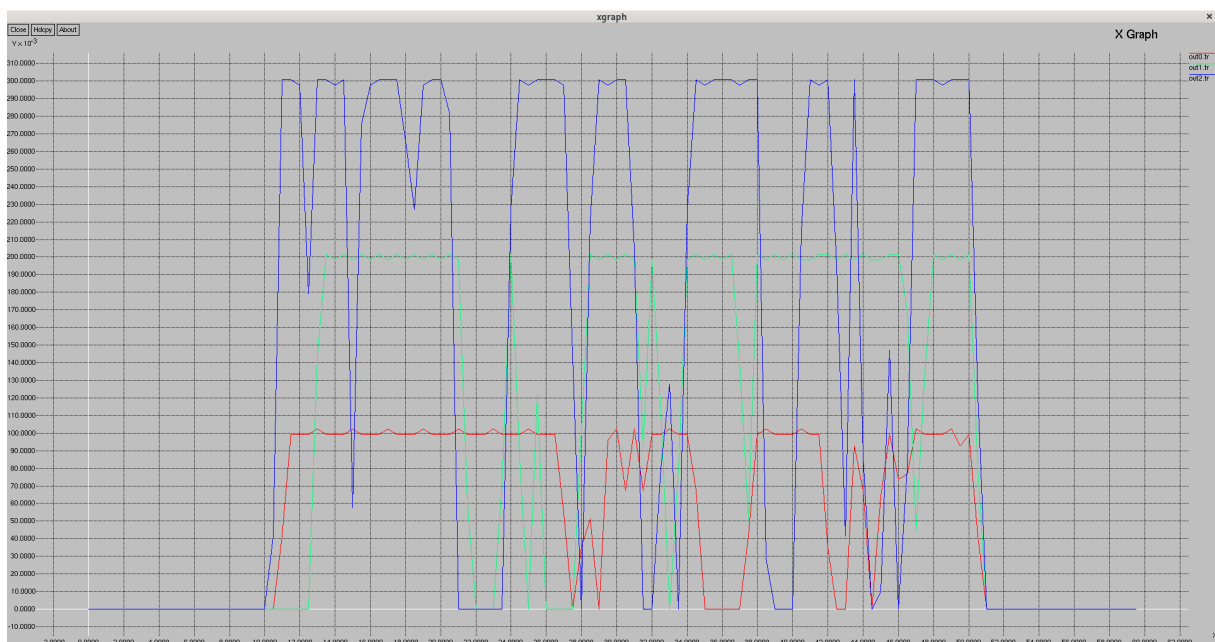


Figure 1: Xgraph output

1.2 Explanation of the custom trace file

```
1 0 0.0
2 0.5 0.0
3 1 0.0
4 1.5 0.0
5 2 0.0
6 2.5 0.0
7 3 0.0
```

8	3.5	0.0
9	4	0.0
10	4.5	0.0
11	5	0.0
12	5.5	0.0
13	6	0.0
14	6.5	0.0
15	7	0.0
16	7.5	0.0
17	8	0.0
18	8.5	0.0
19	9	0.0
20	9.5	0.0
21	10	0.0
22	10.5	0.0
23	11	0.041599999999999998
24	11.5	0.099199999999999997
25	12	0.099199999999999997
26	12.5	0.099199999999999997
27	13	0.1024
28	13.5	0.099199999999999997
29	14	0.099199999999999997
30	14.5	0.099199999999999997

Explanation: The trace file *out0.tr* contains two columns: the first column is the simulation time in seconds, and the second column is the measured throughput at that time. For example, at 11 seconds, the throughput is 0.0416 Mbps, and at 13 seconds, it is 0.1024 Mbps. Initially, the throughput is 0 because the flow has not started. As the simulation progresses, the throughput increases and fluctuates slightly due to packet transmission intervals.

1.3 Add NAM trace

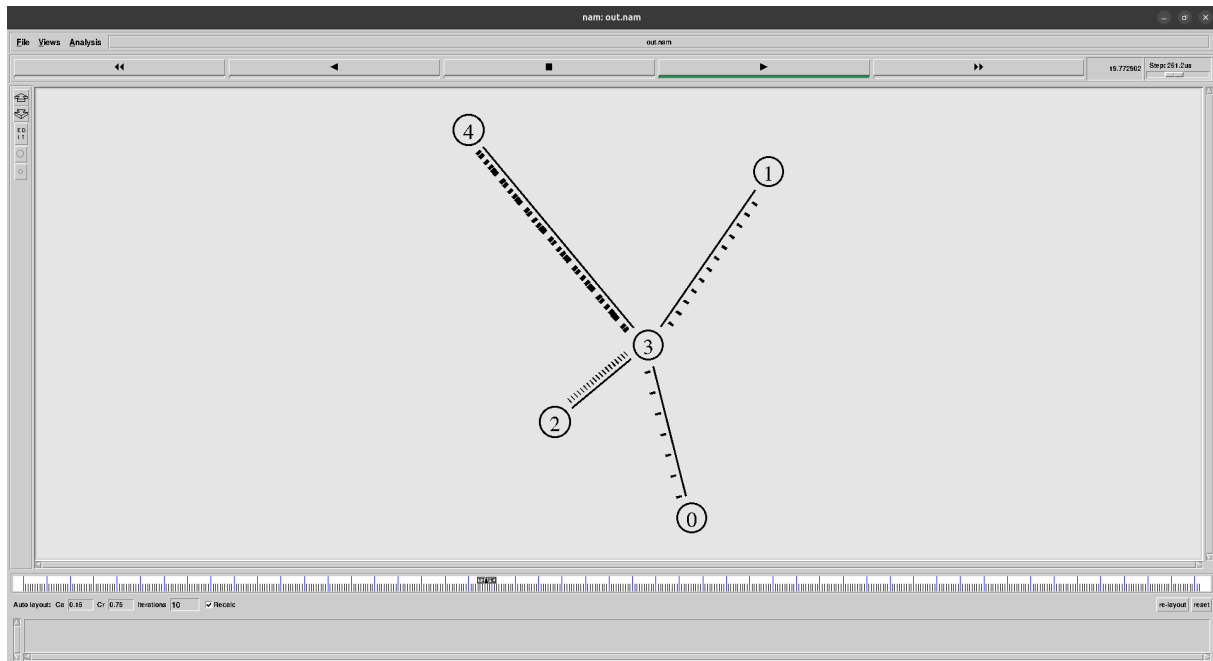


Figure 2: NAM trace

Modified code:

```
1  #Create a simulator object
2  set ns [new Simulator]
3
4  # 1.c)
5  # Create a NAM trace file
6  set nf [open out.nam w]
7  $ns namtrace-all $nf
8  # 1.c)
9
10 #Open the output files
11 set f0 [open out0.tr w]
12 set f1 [open out1.tr w]
13 set f2 [open out2.tr w]
14
15 #Create 5 nodes
16 set n0 [$ns node]
17 set n1 [$ns node]
18 set n2 [$ns node]
19 set n3 [$ns node]
20 set n4 [$ns node]
21
22 #Connect the nodes
23 $ns duplex-link $n0 $n3 1Mb 100ms DropTail
24 $ns duplex-link $n1 $n3 1Mb 100ms DropTail
```

```

25 $ns duplex-link $n2 $n3 1Mb 100ms DropTail
26 $ns duplex-link $n3 $n4 1Mb 100ms DropTail
27
28 #Define a 'finish' procedure
29 proc finish {} {
30     global f0 f1 f2 ns nf
31     #Close the Xgraph output files
32     close $f0
33     close $f1
34     close $f2
35
36     $ns flush-trace
37     # Close the NAM file
38     close $nf
39     #Call xgraph to display the results
40     exec xgraph out0.tr out1.tr out2.tr -geometry 800x400 &
41     # Launch NAM for animation
42     exec nam out.nam &
43     exit 0
44 }
45
46
47 #Define a procedure that attaches a UDP agent to a
    previously created node
48 #'node' and attaches an Expoo traffic generator to the agent
    with the
49 #characteristic values 'size' for packet size 'burst' for
    burst time,
50 #'idle' for idle time and 'rate' for burst peak rate. The
    procedure connects
51 #the source with the previously defined traffic sink 'sink'
    and returns the
52 #source object.
53 proc attach-expoo-traffic { node sink size burst idle rate }
    {
54     #Get an instance of the simulator
55     set ns [Simulator instance]
56
57     #Create a UDP agent and attach it to the node
58     set source [new Agent/UDP]
59     $ns attach-agent $node $source
60
61     #Create an Expoo traffic agent and set its configuration
        parameters
62     set traffic [new Application/Traffic/Exponential]
63     $traffic set packetSize_ $size
64     $traffic set burst_time_ $burst
65     $traffic set idle_time_ $idle

```

```

66   $traffic set rate_ $rate
67
68   # Attach traffic source to the traffic generator
69   $traffic attach-agent $source
70   #Connect the source and the sink
71   $ns connect $source $sink
72   return $traffic
73 }
74
75
76   #Define a procedure which periodically records the bandwidth
       received by the
77   #three traffic sinks sink0/1/2 and writes it to the three
       files f0/1/2.
78   proc record {} {
79       global sink0 sink1 sink2 f0 f1 f2
80       #Get an instance of the simulator
81       set ns [Simulator instance]
82       #Set the time after which the procedure should be called
           again
83       set time 0.5
84       #How many bytes have been received by the traffic sinks?
85       set bw0 [$sink0 set bytes_]
86       set bw1 [$sink1 set bytes_]
87       set bw2 [$sink2 set bytes_]
88       #Get the current time
89       set now [$ns now]
90       #Calculate the bandwidth (in MBit/s) and write it to the
           files
91       puts $f0 "$now [expr $bw0/$time*8/1000000]"
92       puts $f1 "$now [expr $bw1/$time*8/1000000]"
93       puts $f2 "$now [expr $bw2/$time*8/1000000]"
94       #Reset the bytes_ values on the traffic sinks
95       $sink0 set bytes_ 0
96       $sink1 set bytes_ 0
97       $sink2 set bytes_ 0
98       #Re-schedule the procedure
99       $ns at [expr $now+$time] "record"
100  }
101
102
103   #Create three traffic sinks and attach them to the node n4
104   set sink0 [new Agent/LossMonitor]
105   set sink1 [new Agent/LossMonitor]
106   set sink2 [new Agent/LossMonitor]
107   $ns attach-agent $n4 $sink0
108   $ns attach-agent $n4 $sink1
109   $ns attach-agent $n4 $sink2

```

```

110
111 #Create three traffic sources
112 set source0 [attach-expoo-traffic $n0 $sink0 200 2s 1s 100k]
113 set source1 [attach-expoo-traffic $n1 $sink1 200 2s 1s 200k]
114 set source2 [attach-expoo-traffic $n2 $sink2 200 2s 1s 300k]
115
116 #Start logging the received bandwidth
117 $ns at 0.0 "record"
118 #Start the traffic sources
119 $ns at 10.0 "$source0 start"
120 $ns at 10.0 "$source1 start"
121 $ns at 10.0 "$source2 start"
122 #Stop the traffic sources
123 $ns at 50.0 "$source0 stop"
124 $ns at 50.0 "$source1 stop"
125 $ns at 50.0 "$source2 stop"
126 #Call the finish procedure after 60 seconds simulation time
127 $ns at 60.0 "finish"
128
129 #Run the simulation
130 $ns run

```

Exercise 2: Two Competing CBR Flows

2.1 Two Screenshots of Simulation

Calculation for Interval value:

Packet size = 500 bytes = $500 \times 8 = 4000$ bits

Rate = 800 Kbps = $800 \times 1000 = 800,000$ bps

Interval = $\frac{\text{Packet size}}{\text{Rate}} = \frac{4000}{800000} = 0.005$ seconds

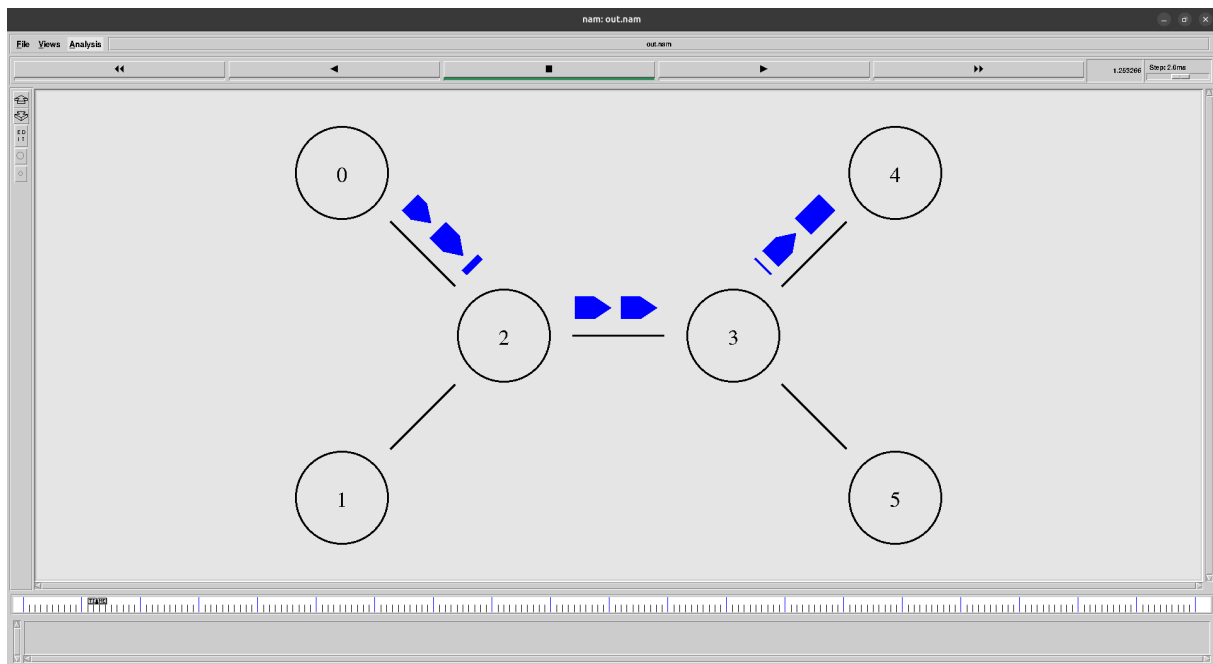


Figure 3: Single CBR Flow

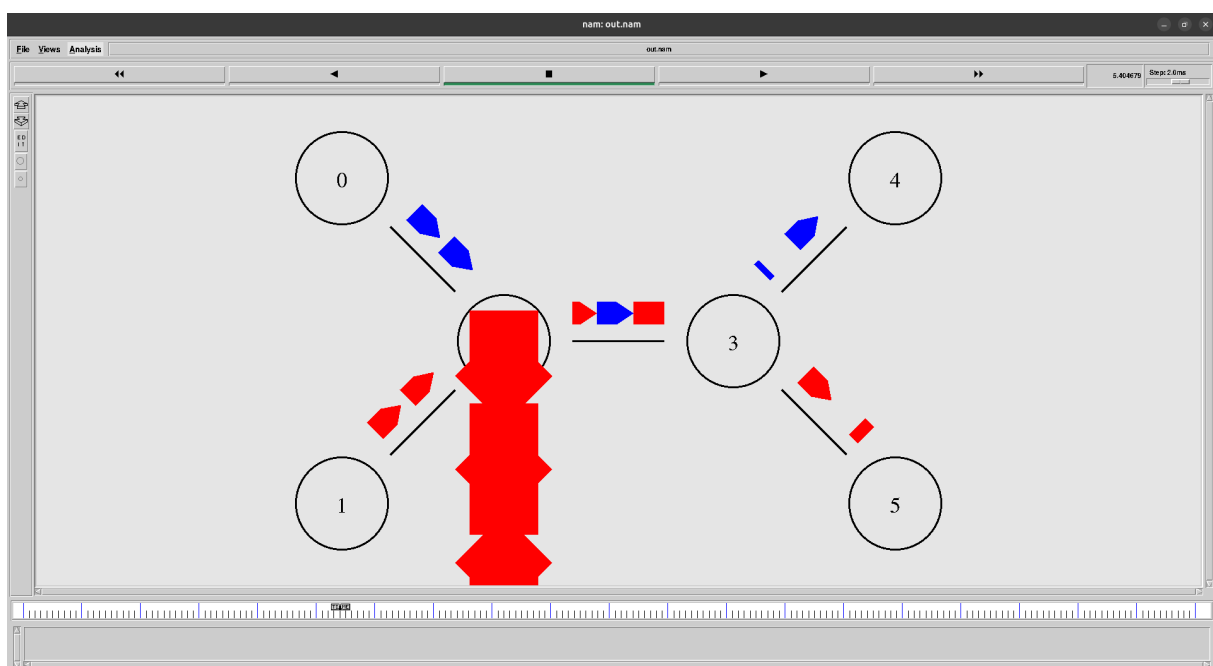


Figure 4: Double CBR Flow

2.2 Throughput Analysis

Modified code:

```

1  # =====
2  # Exercise 2(b): Two Competing CBR Flows - Throughput
   # Measurement
3  # =====

```

```

4
5 # Create simulator
6 set ns [new Simulator]
7
8 # Define colors for flows (this affects NAM visualization)
9 $ns color 0 Red
10 $ns color 1 Blue
11
12 # Create NAM trace file
13 set nf [open out.nam w]
14 $ns namtrace-all $nf
15
16 # Create Xgraph output files
17 set f0 [open out0.tr w]
18 set f1 [open out1.tr w]
19
20 # =====
21 # Create nodes
22 # =====
23 set n0 [$ns node]
24 set n1 [$ns node]
25 set n2 [$ns node]
26 set n3 [$ns node]
27 set n4 [$ns node]
28 set n5 [$ns node]
29
30 # =====
31 # Connect nodes (1Mbps, 10ms, DropTail)
32 # =====
33 $ns duplex-link $n0 $n2 1Mb 10ms DropTail
34 $ns duplex-link $n1 $n2 1Mb 10ms DropTail
35 $ns duplex-link $n2 $n3 1Mb 10ms DropTail
36 $ns duplex-link $n3 $n4 1Mb 10ms DropTail
37 $ns duplex-link $n3 $n5 1Mb 10ms DropTail
38
39 # Orient links for better visualization
40 $ns duplex-link-op $n0 $n2 orient right-down
41 $ns duplex-link-op $n1 $n2 orient right-up
42 $ns duplex-link-op $n2 $n3 orient right
43 $ns duplex-link-op $n3 $n4 orient right-up
44 $ns duplex-link-op $n3 $n5 orient right-down
45
46 # =====
47 # Create traffic sinks
48 # =====
49 set sink0 [new Agent/LossMonitor]
50 $ns attach-agent $n4 $sink0
51

```



```

52 set sink1 [new Agent/LossMonitor]
53 $ns attach-agent $n5 $sink1
54
55 # =====
56 # Create UDP agents and CBR sources
57 # =====
58 # Flow 0: n0 -> n4
59 set udp0 [new Agent/UDP]
60 $udp0 set class_ 0
61 $ns attach-agent $n0 $udp0
62 set cbr0 [new Application/Traffic/CBR]
63 $cbr0 set packetSize_ 500
64 $cbr0 set interval_ 0.005 ;# 800 Kbps
65 $cbr0 attach-agent $udp0
66 $ns connect $udp0 $sink0
67
68 # Flow 1: n1 -> n5
69 set udp1 [new Agent/UDP]
70 $udp1 set class_ 1
71 $ns attach-agent $n1 $udp1
72 set cbr1 [new Application/Traffic/CBR]
73 $cbr1 set packetSize_ 500
74 $cbr1 set interval_ 0.005 ;# 800 Kbps
75 $cbr1 attach-agent $udp1
76 $ns connect $udp1 $sink1
77
78 # =====
79 # Record throughput for Xgraph
80 # =====
81 proc record {} {
82     global ns sink0 sink1 f0 f1
83     set time 0.5
84     set now [$ns now]
85
86     set bw0 [$sink0 set bytes_]
87     set bw1 [$sink1 set bytes_]
88     puts $f0 "$now [expr $bw0/$time*8/1000000]"
89     puts $f1 "$now [expr $bw1/$time*8/1000000]"
90
91     $sink0 set bytes_ 0
92     $sink1 set bytes_ 0
93     $ns at [expr $now+$time] "record"
94 }
95
96 $ns at 0.0 "record"
97
98 # =====
99 # Flow start/stop times

```

```

100  # =====
101  $ns at 1.0 "$cbr0 start"
102  $ns at 20.0 "$cbr0 stop"
103
104  $ns at 5.0 "$cbr1 start"
105  $ns at 15.0 "$cbr1 stop"
106
107  # =====
108  # Finish routine
109  # =====
110  proc finish {} {
111      global f0 f1 nf
112      close $f0
113      close $f1
114      close $nf
115
116      exec xgraph out0.tr out1.tr -geometry 1920x1080 &
117      exec nam out.nam &
118      exit 0
119  }
120
121  # Call finish at end of simulation
122  $ns at 25.0 "finish"
123
124  # =====
125  # Run simulation
126  # =====
127  $ns run

```



Figure 5: Xgraph output

Comment:

When only flow n0–n4 was active (1 s – 5 s), the measured throughput was about 0.8 Mbps with no packet loss. When flow n1–n5 started at 5 s, both CBR sources jointly offered 1.6 Mbps to the 1 Mbps bottleneck link, causing severe congestion. The earlier flow (n0–n4) maintained nearly full throughput, while the later flow (n1–n5) experienced heavy losses. The total delivered throughput of ≈ 1 Mbps matches the bottleneck capacity. This illustrates how UDP traffic without congestion control can create unfair sharing and packet drops on shared links.

2.3 Packet Drop Analysis

Flow 0 (started first):

Received 3805 packets

Average throughput: ≈ 0.8 Mbps overall, ≈ 0.64 Mbps during competition

Experienced: 0 drops

Flow 1 (started later):

Received only 523 packets

Average throughput: 0.21 Mbps during competition

Experienced: 1492 drops (100% of all drops)

The Unfairness is Even Worse Than Expected:

Not only does Flow 1 suffer all the drops, but it also:

Gets only 25% of the bandwidth it should get

While Flow 0 maintains 80% of its target bandwidth

3:1 bandwidth ratio instead of the expected 1:1

Why This Happens:

The DropTail queue + timing synchronization creates a vicious cycle:

Flow 0's packets arrive first → get queued

Flow 1's packets arrive second → queue full → dropped

Flow 0 packets get transmitted → make space in queue

Next round: Flow 0 packets again arrive first → repeat

Flow 1 can only squeeze in packets during rare moments when its timing happens to beat Flow 0, which is why it gets 500 packets instead of 0, but nowhere near the 1600 it should get.

Exercise 3: CBR flow competing with FTP over TCP flow

3.1 Two Screenshots of Simulation

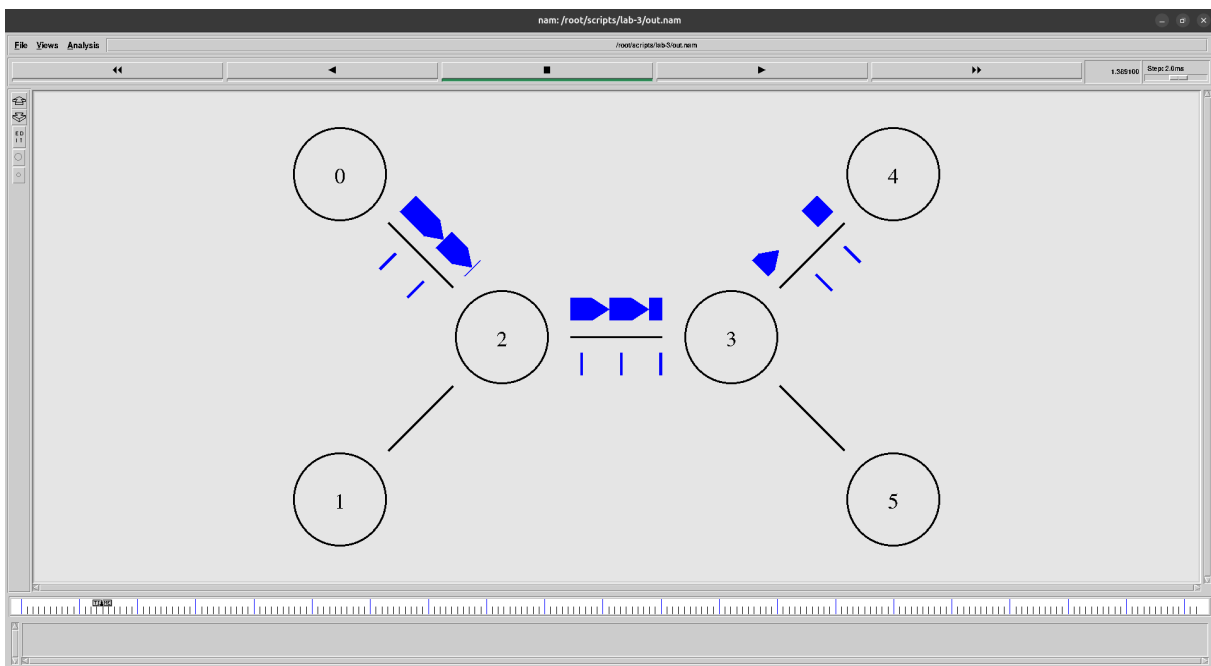


Figure 6: Single Flow

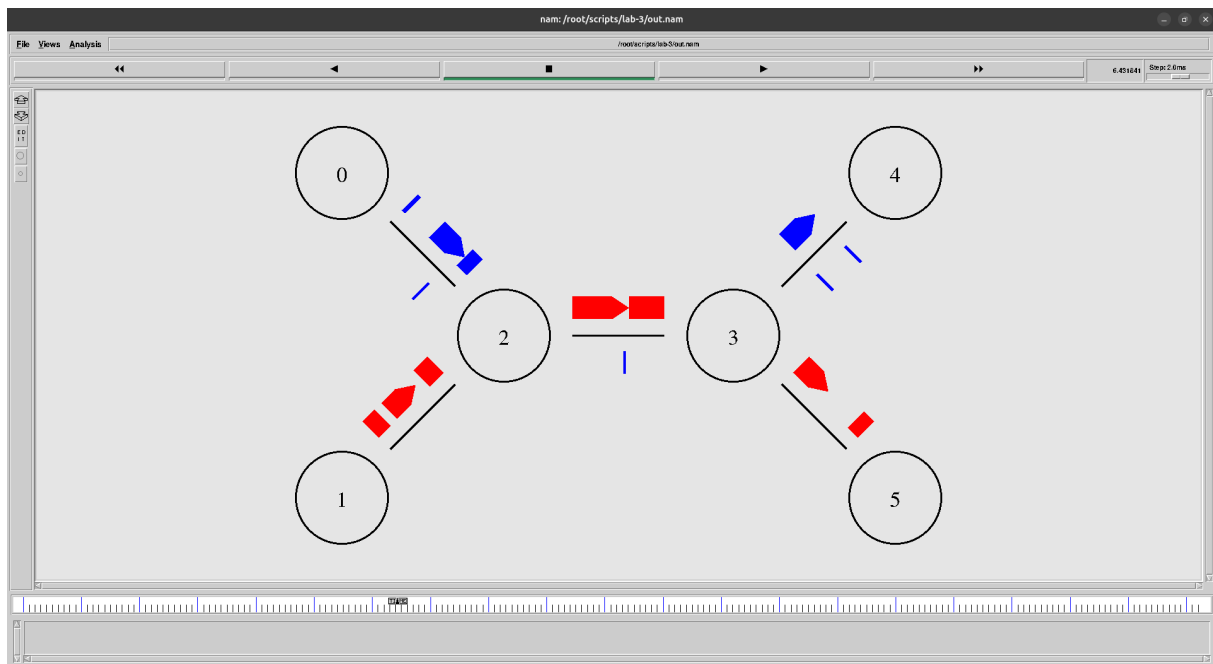


Figure 7: Double Flow

3.2 Throughput Analysis

Modified code:

```

1  # Exercise 3(b): TCP/FTP competing with CBR/UDP
2
3  # Create simulator
4  set ns [new Simulator]
5
6  # Define colors for flows (this affects NAM visualization)
7  $ns color 0 Blue;    # TCP
8  $ns color 1 Red;     # CBR
9
10 # Create NAM trace file
11 set nf [open out.nam w]
12 $ns namtrace-all $nf
13
14 # Create trace file for analysis
15 set tracefile [open out.tr w]
16 $ns trace-all $tracefile
17
18 # Create Xgraph output files
19 set f0 [open out0.tr w];    # TCP
20 set f1 [open out1.tr w];    # CBR
21
22 # Create nodes
23 set n0 [$ns node]
24 set n1 [$ns node]
25 set n2 [$ns node]
```

```

26 set n3 [$ns node]
27 set n4 [$ns node]
28 set n5 [$ns node]
29
30 # Connect nodes (1Mbps, 10ms, DropTail)
31 $ns duplex-link $n0 $n2 1Mb 10ms DropTail
32 $ns duplex-link $n1 $n2 1Mb 10ms DropTail
33 $ns duplex-link $n2 $n3 1Mb 10ms DropTail
34 $ns duplex-link $n3 $n4 1Mb 10ms DropTail
35 $ns duplex-link $n3 $n5 1Mb 10ms DropTail
36
37 # Orient links for better visualization
38 $ns duplex-link-op $n0 $n2 orient right-down
39 $ns duplex-link-op $n1 $n2 orient right-up
40 $ns duplex-link-op $n2 $n3 orient right
41 $ns duplex-link-op $n3 $n4 orient right-up
42 $ns duplex-link-op $n3 $n5 orient right-down
43
44
45 # Flow 0: TCP / FTP (n0 -> n4)
46 set tcp0 [new Agent/TCP]
47 $tcp0 set class_ 0
48 $tcp0 set window_ 100
49 $tcp0 set packetSize_ 500
50 $ns attach-agent $n0 $tcp0
51
52 set sink0 [new Agent/TCPSink]
53 $ns attach-agent $n4 $sink0
54
55 $ns connect $tcp0 $sink0
56
57 set ftp0 [new Application/FTP]
58 $ftp0 attach-agent $tcp0
59
60
61 # Flow 1: CBR / UDP (n1 -> n5)
62 set udp1 [new Agent/UDP]
63 $udp1 set class_ 1
64 $ns attach-agent $n1 $udp1
65
66 set sink1 [new Agent/LossMonitor]
67 $ns attach-agent $n5 $sink1
68
69 $ns connect $udp1 $sink1
70
71 set cbr1 [new Application/Traffic/CBR]
72 $cbr1 set packetSize_ 500
73 $cbr1 set interval_ 0.005 ;# 800 Kbps

```

```

74 $cbr1 attach-agent $udp1
75
76
77 # Record throughput for Xgraph
78 proc record {} {
79     global ns f0 f1 sink0 sink1
80
81     set timeInterval 0.5
82     set now [$ns now]
83
84     # Calculate throughput (Mbps)
85     set bw0 [expr [$sink0 set bytes_]*8.0/($timeInterval
86         *1000000)]
87     set bw1 [expr [$sink1 set bytes_]*8.0/($timeInterval
88         *1000000)]
89
90     puts $f0 "$now $bw0"
91     puts $f1 "$now $bw1"
92
93     # Reset counters for next interval
94     $sink0 set bytes_ 0
95     $sink1 set bytes_ 0
96
97     $ns at [expr $now+$timeInterval] "record"
98 }
99
100 $ns at 0.0 "record"
101
102 # Flow start/stop times
103 # n0-n4
104 $ns at 1.0 "$ftp0 start"
105 $ns at 20.0 "$ftp0 stop"
106 # n1-n5
107 $ns at 5.0 "$cbr1 start"
108 $ns at 15.0 "$cbr1 stop"
109
110 # Finish routine
111 proc finish {} {
112     global ns nf f0 f1 tracefile
113
114     $ns flush-trace
115     close $tracefile
116     close $f0
117     close $f1
118     close $nf
119
120     exec xgraph out0.tr out1.tr -geometry 1920x1080 -x "Time (s"

```

```

120     )" -y "Throughput (Mbps)" &
121     exec nam out.nam &
122     exit 0
123 }
124 # Call finish at end of simulation
125 $ns at 25.0 "finish"
126
127 # Run simulation
128 $ns run

```

3.2.1 Throughput Analysis Results



Figure 8: Xgraph screenshot

Single Flow Transmission Periods:

Period 1.0-5.0 seconds (Only TCP/FTP active):

TCP Throughput: ≈ 0.95 -1.0 Mbps

Observation: TCP achieves nearly 100% utilization of the 1 Mbps bottleneck link when operating alone, demonstrating efficient congestion control and window management.

Period 15.0-20.0 seconds (Only TCP/FTP active):

TCP Throughput: 0.95-1.0 Mbps

Observation: TCP quickly recovers full link utilization after the CBR flow terminates at 15 seconds.

Competing Flows Period (5.0-15.0 seconds):

Individual Flow Throughput:

CBR/UDP Flow: Maintains constant ≈ 0.8 Mbps (as configured)

TCP/FTP Flow: Reduced ≈ 0.2 Mbps

Observation: The CBR flow dominates due to its aggressive, non-responsive nature

Cumulative Throughput:

Total: ≈ 1.0 Mbps

Observation: The bottleneck link remains fully utilized, demonstrating efficient resource sharing despite the competition.

Key Observations from Throughput Graphs

TCP Adaptive Behavior:

- Shows significant throughput variation and reduction when competing with CBR
- Demonstrates congestion avoidance by backing off transmission rate
- Quickly recovers when competition ceases

CBR Non-Responsive Nature:

- Maintains constant 0.8 Mbps rate regardless of network conditions
- Does not adapt to congestion, causing unfair bandwidth sharing

Link Utilization Efficiency:

- The bottleneck link maintains $\approx 100\%$ utilization throughout simulation
- No significant bandwidth wastage observed

Fairness Issues:

- CBR/UDP consumes 80% of available bandwidth while TCP adapts to use only 20%
- This demonstrates the inherent unfairness when responsive TCP competes with non-responsive UDP traffic.

The throughput graphs clearly illustrate the fundamental differences between congestion-responsive (TCP) and non-responsive (CBR/UDP) traffic, highlighting important considerations for network protocol design and quality of service management.

3.3 Packet Drop Analysis

To analyze packet drops from the trace file, I used the following approach:

```
1  # Count total dropped packets
2  grep "\^d" out.tr | wc -l
3
4  # Count TCP dropped packets
5  grep "\^d.*tcp" out.tr | wc -l
6
7  # Count CBR dropped packets
8  grep "\^d.*cbr" out.tr | wc -l
```

Packet Drop Results:

Total Dropped Packets:

Overall 106 packets were dropped during the simulation.

Flow-Specific Packet Drops:

TCP/FTP Flow (n0-n4): 60 packets dropped (56.6% of total drops)

CBR/UDP Flow (n1-n5): 46 packets dropped (43.4% of total drops)

Analysis and Comments:

During the period when only the TCP/FTP flow was active (1 s – 5 s), no packet drops occurred since the link bandwidth was sufficient. When the UDP/CBR flow started at 5 s, congestion built up at the bottleneck link (n2–n3). The queue overflowed, resulting in a total of 106 packet drops. About 56.6% of the dropped packets belonged to the TCP flow, while 43.4% of UDP packets were lost. This shows that the non-adaptive UDP source consumed most of the bandwidth and forced TCP to back off.

3.4 Comment on Exercise 2 and Exercise 3 Simulation Result

In Exercise 2, both UDP/CBR flows transmitted at fixed rates exceeding the bottleneck capacity, leading to persistent congestion and numerous packet drops for both flows. In contrast, Exercise 3 introduced a TCP flow competing with a CBR flow. Because TCP implements congestion control, it reduced its sending rate whenever losses occurred, while the UDP/CBR source continued transmitting at a constant rate. As a result, the total number of dropped packets decreased in Exercise 3 compared with Exercise 2. However, most of the remaining drops were experienced by the TCP flow, which yielded bandwidth to the unresponsive UDP source. The cumulative throughput during the overlap period remained close to the link capacity ≈ 1 Mbps, confirming full utilization but poor fairness.

References

- [1] Mark Greis, Tutorial for the Network Simulator "ns": <http://www.isi.edu/nsnam/ns/tutorial/index.html>
- [2] GNU AWK User's Guide: <https://www.gnu.org/software/gawk/manual/gawk.html>